

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTAD DE INGENIERÍA
Ingeniería de Sistemas

Sistema de Gestión Operativa de un Hotel

Hotel Imperial — Sistema de Información

Bases de Datos Avanzadas

Proyecto Final del Semestre · 2026

Integrantes del Equipo

Juan David Amaya Patiño jdamayap@udistrital.edu.co

Juan Pablo Mosquera Marín jpmosqueram@udistrital.edu.co

Código Juan David: 20221020057

Código Juan Pablo: 20221020026

Grupo: 020-82

Docente: René Alejandro Lobo Quintero

Fecha de entrega: Mayo de 2026

Índice

1. Descripción del Sistema	3
1.1. ¿Qué problema resuelve?	3
1.2. Usuarios del Sistema	3
1.3. Funcionalidades Principales	3
2. Casos de Uso	4
3. Diagrama Entidad-Relación	6
3.1. Diagrama ER	6
4. Diccionario de Datos	7
5. Requerimientos Técnicos Implementados	10
5.1. R1 · Base de Datos Relacional	11
5.2. R2 · CRUD Completo	11
5.3. R3 · Procedimiento Almacenado / Función	12
5.4. R4 · Triggers	12
5.5. R5 · Subconsulta	12
5.6. R6 · Vista (VIEW)	12
5.7. R7 · Índices	13
5.8. R8 · Transacciones	13
6. Endpoints Principales de la API	13

1. Descripción del Sistema

1.1. ¿Qué problema resuelve?

El sistema gestiona el ciclo operativo completo de un establecimiento de hospedaje, resolviendo principalmente el problema de la **dispersión de datos** al integrar la reserva de habitaciones con el consumo de servicios adicionales. El proceso permite vincular a un cliente específico con una habitación durante un periodo de tiempo determinado, mientras registra de manera simultánea cualquier solicitud de servicio extra que el huésped realice durante su estancia.

Además, el sistema organiza la estructura administrativa al clasificar al personal por áreas y cargos, asignando responsabilidades directas sobre los servicios prestados. En conjunto, el modelo garantiza la **trazabilidad total** del consumo del cliente, desde su registro inicial y datos de contacto hasta la facturación final de los costos asociados a su reserva y los servicios atendidos por los empleados.

La interfaz de interacción es una **aplicación web** conectada a una API REST desarrollada en Spring Boot, con base de datos PostgreSQL.

1.2. Usuarios del Sistema

Tipo de Usuario	Acciones y consultas que realiza
Recepción	Registra y actualiza datos de clientes (teléfonos, correos). Consulta la disponibilidad de habitaciones. Crea y modifica reservas de alojamiento. Consulta los servicios disponibles. Registra solicitudes de servicios adicionales vinculadas a reservas activas, identificándose con su cédula de empleado.
Administración	Gestión total del personal y áreas de trabajo. Configura el catálogo de servicios y sus precios. Consulta la auditoría de cambios sobre empleados.

Cuadro 1: Tipos de usuarios del sistema y sus responsabilidades

1.3. Funcionalidades Principales

- F1.** Registro y consulta de clientes con datos de contacto multivalorados (correos en `CorreoCliente` y teléfonos en `TelefonoCliente`).
- F2.** Gestión del catálogo de habitaciones: creación, consulta por id y actualización de tipo, precio y disponibilidad.

- F3.** Creación y modificación de reservas en **Reservar**, vinculando cliente–habitación–fechas, con columna calculada **fecha_rango**.
- F4.** Registro de solicitudes de servicios adicionales en **Solicitar**, asociadas a una reserva activa y al empleado responsable de la atención.
- F5.** Gestión de empleados, áreas de trabajo y catálogo de servicios.
- F6.** Validación automática de solapamiento de reservas para la misma habitación mediante el trigger **trg_no_solapamiento** (BEFORE INSERT OR UPDATE en **Reservar**).
- F7.** Auditoría automática de cambios (INSERT, UPDATE, DELETE) en la tabla **Empleado** mediante el trigger **trg_audit_empleado**, almacenando los valores anteriores y nuevos en formato JSONB.

2. Casos de Uso

A continuación se presentan los casos de uso principales del sistema, en formato de tabla conforme a los lineamientos del proyecto.

CU-01 · Registrar Reserva	
Actor	Recepción
Descripción	El recepcionista selecciona un cliente registrado y una habitación disponible, define fechas de llegada y salida, y crea la reserva mediante POST /reservas . El trigger trg_no_solapamiento valida automáticamente que no existan reservas con fechas superpuestas para la misma habitación antes de confirmar el INSERT.
Precondición	El cliente y la habitación deben estar registrados. La fecha de salida debe ser posterior a la de llegada (CHECK en la tabla Reservar).
Postcondición	La reserva queda registrada en Reservar con su fecha_rango calculado automáticamente.

Cuadro 2: Caso de uso CU-01: Registrar Reserva

CU-02 · Registrar Solicitud de Servicio

Actor	Personal de Servicio
Descripción	El empleado asocia un servicio del catálogo a una reserva activa mediante <code>POST /reservas/solicitar</code> , indicando el nombre de la solicitud, la fecha, la hora y su propia cédula como responsable de la atención.
Precondición	Debe existir una reserva activa (<code>IdReserva</code> válido), el servicio debe estar en el catálogo (<code>IdServicio</code> válido) y el empleado debe estar registrado en <code>Empleado</code> (<code>Cedula</code> válida).
Postcondición	La solicitud queda registrada en <code>Solicitar</code> , vinculada a la reserva, al servicio y al empleado.

Cuadro 3: Caso de uso CU-02: Registrar Solicitud de Servicio

CU-03 · Consultar Disponibilidad de Habitaciones

Actor	Recepción / Personal de Servicio
Descripción	El usuario consulta el catálogo de habitaciones mediante <code>GET /habitaciones</code> o <code>GET /habitaciones/{id}</code> para verificar tipo, precio y disponibilidad de una habitación específica.
Precondición	Ninguna (consulta de solo lectura).
Postcondición	El sistema retorna la lista de habitaciones o el detalle de una habitación con su estado de <code>Disponibilidad</code> .

Cuadro 4: Caso de uso CU-03: Consultar Disponibilidad de Habitaciones

CU-04 · Registrar Empleado

Actor	Administración
Descripción	El administrador crea un nuevo registro de empleado con sus datos personales, cargo, área asignada y salario mediante <code>POST /empleados</code> . El trigger <code>trg_audit_empleado</code> registra automáticamente en <code>Empleado_audit</code> la operación 'I', el usuario de base de datos, la marca de tiempo y el valor nuevo en formato JSONB.
Precondición	El área referenciada debe existir en <code>Area</code> . La cédula debe tener entre 8 y 10 dígitos numéricos.
Postcondición	El empleado queda registrado y el INSERT queda auditado en <code>Empleado_audit</code> .

Cuadro 5: Caso de uso CU-04: Registrar Empleado

CU-05 · Actualizar Reserva

Actor	Recepción
Descripción	El recepcionista modifica los datos de una reserva existente mediante <code>PUT /reservas/{id}</code> . El trigger <code>trg_no_solapamiento</code> también se activa en el <code>UPDATE</code> para garantizar que las nuevas fechas no generen conflicto con otras reservas.
Precondición	La reserva debe existir (<code>IdReserva</code> válido).
Postcondición	La reserva queda actualizada en <code>Reservar</code> con la nueva <code>fecha_rango</code> calculada.

Cuadro 6: Caso de uso CU-05: Actualizar Reserva

CU-06 · Gestionar Catálogo de Servicios

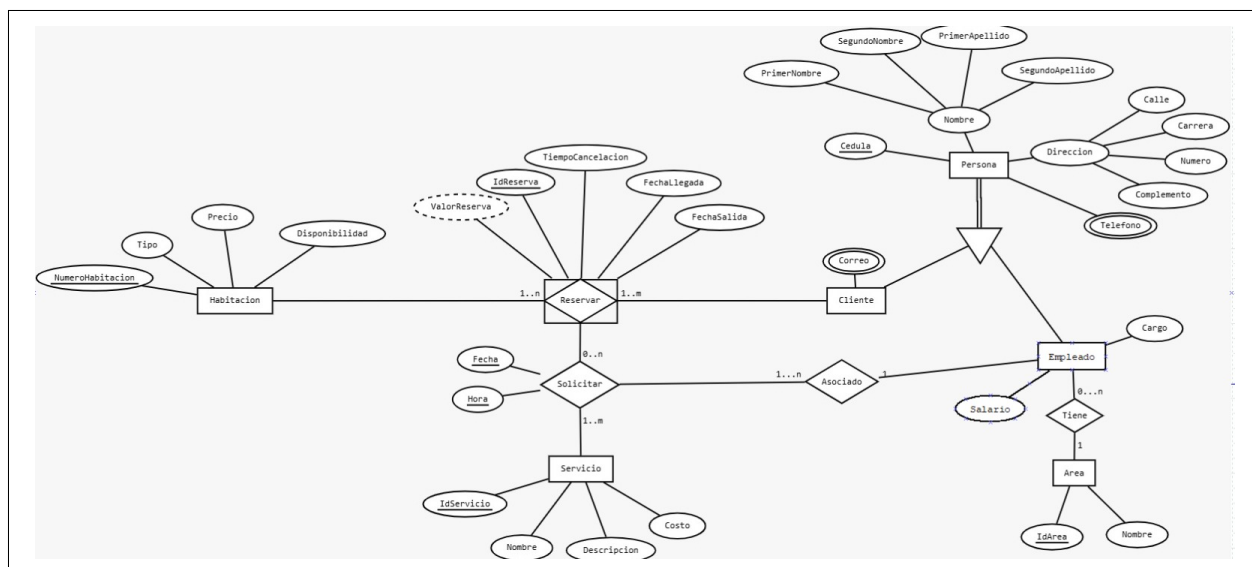
Actor	Administración
Descripción	El administrador crea un nuevo servicio en la tabla <code>Servicio</code> (desayuno, lavandería, etc.) mediante <code>POST /empleados/servicios</code> , definiendo nombre único, descripción y costo.
Precondición	El nombre del servicio no debe existir previamente (restricción <code>UNIQUE</code> en <code>NombreServicio</code>).
Postcondición	El servicio queda registrado y disponible en el catálogo.

Cuadro 7: Caso de uso CU-06: Gestionar Catálogo de Servicios

3. Diagrama Entidad-Relación

El modelo relacional implementado se compone de **diez tablas**, con llaves primarias, foráneas y restricciones `CHECK` / `NOT NULL` relevantes, superando el mínimo de cinco requerido. Requiere la extensión `btree_gist` de PostgreSQL para soportar el tipo `daterange` y el operador de solapamiento `&&`.

3.1. Diagrama ER



Las cardinalidades del modelo son las siguientes:

- **Cliente — Reservar:** 1:N (un cliente puede tener varias reservas).
- **Habitacion — Reservar:** 1:N (una habitación puede tener varias reservas en distintos periodos, sin solapamiento).
- **Reservar — Servicio:** N:M (una reserva puede solicitar varios servicios; un servicio puede aparecer en varias reservas), resuelta con la tabla **Solicitar**.
- **Empleado — Solicitar:** 1:N (un empleado puede atender varias solicitudes; cada solicitud registra la cédula del empleado responsable).
- **Empleado — Area:** N:1 (varios empleados pertenecen a un área).
- **Cliente — TelefonoCliente / CorreoCliente:** 1:N (multivalorado).
- **Empleado — TelefonoEmpleado:** 1:N (multivalorado).

4. Diccionario de Datos

Se describen a continuación las tablas principales del sistema con el nombre, tipo de dato, restricciones y propósito de cada columna.

Tabla: Cliente

Columna	Tipo	Restricción	Propósito
Cedula	varchar(10)	PK, NOT NULL, CHECK (regex)	Número de cédula del cliente. Debe contener entre 8 y 10 dígitos numéricos. Identifica unívocamente al cliente.
PrimerNombre	varchar(255)	NOT NULL	Primer nombre del cliente. Obligatorio para identificación.
SegundoNombre	varchar(255)	—	Segundo nombre del cliente. Opcional.
PrimerApellido	varchar(255)	NOT NULL	Primer apellido del cliente. Obligatorio.
SegundoApellido	varchar(255)	—	Segundo apellido del cliente. Opcional.
Calle	varchar(255)	NOT NULL	Componente de la dirección: nombre o número de la calle.
Carrera	varchar(255)	NOT NULL	Componente de la dirección: número de la carrera.
Numero	varchar(255)	NOT NULL	Número de la propiedad en la dirección.
Complemento	varchar(255)	NOT NULL	Información adicional de la dirección (apto, casa, piso, etc.).

Cuadro 8: Diccionario de datos — tabla **Cliente**

Tabla: Habitacion

Columna	Tipo	Restricción	Propósito
NumeroHabitacion	bigint	PK, NOT NULL, UNIQUE	Número identificador único de la habitación. Clave principal del catálogo de habitaciones.
Tipo	varchar(255)	NOT NULL, CHECK	Categoría de la habitación. Solo acepta los valores Sencilla , Doble o Suite .
Precio	DECIMAL(10,2)	NOT NULL, CHECK (≥ 0)	Precio por noche de la habitación en la moneda local. No puede ser negativo.
Disponibilidad	boolean	NOT NULL	Indica si la habitación está libre (TRUE) u ocupada (FALSE). Se actualiza manualmente al crear o cancelar una reserva.

Cuadro 9: Diccionario de datos — tabla **Habitacion**

Tabla: Reservar

Columna	Tipo	Restricción	Propósito
IdReserva	bigint	PK, IDENTITY	Identificador único de la reserva, generado automáticamente por la base de datos.
TiempoCancelacion	bigint	NOT NULL	Plazo máximo (en horas o días, según la lógica de negocio) para cancelar la reserva sin penalidad.
FechaLlegada	date	NOT NULL, CHECK	Fecha de inicio de la estadía. Debe ser anterior a FechaSalida.
FechaSalida	date	NOT NULL, CHECK	Fecha de fin de la estadía. Debe ser posterior a FechaLlegada.
Cedula	varchar(10)	NOT NULL, FK Cliente →	Cédula del cliente que realiza la reserva.
NumeroHabitacion	bigint	NOT NULL, FK → Habitación	Número de la habitación reservada.
fecha_rango	daterange	GENERATESTORED	Columna calculada automáticamente como <code>daterange(FechaLlegada, FechaSalida, '[]')</code> . Usada por el trigger <code>trg_no_solapamiento</code> para detectar conflictos de fechas con el operador <code>&&</code> .

Cuadro 10: Diccionario de datos — tabla Reservar

5. Requerimientos Técnicos Implementados

Esta sección describe cómo el sistema satisface cada requerimiento técnico obligatorio definido en el enunciado del proyecto final.

5.1. R1 · Base de Datos Relacional

El esquema cuenta con **diez tablas** con llaves foráneas y las siguientes restricciones:

- CHECK en Habitación.Tipo: solo permite Sencilla, Doble o Suite.
- CHECK en Habitación.Precio: el precio debe ser ≥ 0 .
- CHECK en Servicio.Costo: el costo debe ser ≥ 0 .
- CHECK en Reservar: FechaSalida > FechaLlegada.
- CHECK (regex) en Empleado.Cedula y Cliente.Cedula: formato numérico de 8 a 10 dígitos ($\sim [0-9]\{8,10\}\$$).
- NOT NULL en campos críticos: cédulas, primer nombre, primer apellido, cargo, salario, fechas de reserva, complemento de dirección.
- UNIQUE en NombreArea y NombreServicio.
- Integridad referencial en todas las llaves foráneas.

5.2. R2 · CRUD Completo

Las operaciones CRUD implementadas en los repositorios y controllers de Spring Boot son:

Entidad	Método	URL	Descripción
Cliente	GET	/clientes	Listar todos
	POST	/clientes	Crear cliente
Habitación	GET	/habitaciones	Listar todas
	POST	/habitaciones	Crear habitación
	GET	/habitaciones/{id}	Consultar por número
	PUT	/habitaciones/{id}	Actualizar tipo/precio/disponibilidad
Reservar	GET	/reservas	Listar todas
	POST	/reservas	Crear reserva
	PUT	/reservas/{id}	Actualizar reserva

Cuadro 11: Operaciones CRUD implementadas en los repositorios

Pendiente de implementar: los métodos DELETE para Cliente, Habitacion, Reservar y Empleado, así como PUT para Cliente y Empleado, no están presentes en los controllers ni repositorios actuales.

5.3. R3 · Procedimiento Almacenado / Función

La función `verificar_solapamiento()` en PostgreSQL encapsula la lógica de detección de solapamiento de fechas. Es invocada automáticamente por el trigger `trg_no_solapamiento` antes de cada INSERT o UPDATE en `Reservar`. Utiliza el operador `&&` sobre la columna calculada `fecha_rango` (`daterange`) para detectar intersección entre rangos de fechas. Si se detecta solapamiento, lanza una excepción con `RAISE EXCEPTION` y aborta la operación.

5.4. R4 · Triggers

Se implementan dos triggers con propósito real:

1. **trg_no_solapamiento:** se dispara BEFORE INSERT OR UPDATE en `Reservar` e invoca `verificar_solapamiento()` para garantizar que ninguna habitación tenga dos reservas con fechas superpuestas. Aborta la transacción con `RAISE EXCEPTION` si detecta conflicto. También se activa en UPDATE, protegiendo las modificaciones de reservas existentes.
2. **trg_audit_empleado:** se dispara AFTER INSERT OR UPDATE OR DELETE en `Empleado` e invoca `fn_audit_empleado()` para registrar en `Empleado.audit` el tipo de operación ('I', 'U' o 'D'), el usuario de base de datos (`current_user`), la marca de tiempo y los valores anteriores (`dato_viejo`) y nuevos (`dato_nuevo`) en formato JSONB mediante `row_to_json`.

5.5. R5 · Subconsulta

Pendiente de implementar: la subconsulta de habitaciones con precio superior al promedio no está implementada en ningún endpoint del `HabitacionController` ni en `HabitacionRepository`. Debe agregarse el método correspondiente en el repositorio y el `@GetMapping` en el controller.

5.6. R6 · Vista (VIEW)

La vista `Vista_Cliente_Reserva_Servicios` está definida en el script SQL y consolida en una sola proyección los datos de `Cliente`, `Reservar`, `Solicitar` y `Servicio`.

Pendiente de implementar: ningún endpoint del `ReservaController` ni de ningún otro controller consume esta vista. El método `findAll()` en `ReservarRepository` ejecuta `SELECT * FROM Reservar` directamente, sin usar la vista. Debe implementarse un endpoint que consulte `Vista_Cliente_Reserva_Servicios` para reportes de consumo

por cliente.

5.7. R7 · Índices

Pendiente de implementar: los índices no están presentes en el script SQL de migración. Deben agregarse al migration para que sean aplicados automáticamente al desplegar la base de datos.

5.8. R8 · Transacciones

Pendiente de implementar: el `ReservarRepository` no usa transacciones explícitas (`BEGIN/COMMIT`) ni la anotación `@Transactional` de Spring. Actualmente el `save()` ejecuta solo el `INSERT` en `Reservar`; la actualización de `Disponibilidad` en `Habitacion` no está implementada en ningún método del repositorio ni del controller. Debe implementarse la transacción para garantizar atomicidad entre ambas operaciones.

6. Endpoints Principales de la API

Los siguientes endpoints están implementados en los controllers de Spring Boot. Los marcados con (*pendiente*) corresponden a operaciones definidas en el diseño pero aún no implementadas en el código fuente.

Método	URL	Descripción
GET	/clientes	Listar todos los clientes
POST	/clientes	Registrar nuevo cliente
GET	/clientes/correos	Listar todos los correos
POST	/clientes/correos	Registrar correo de cliente
GET	/clientes/{cedula}/correos	Correos de un cliente por cédula
GET	/clientes/telefonos	Listar todos los teléfonos de clientes
POST	/clientes/telefonos	Registrar teléfono de cliente
GET	/clientes/{cedula}/telefonos	Teléfonos de un cliente por cédula
PUT	/clientes/{cedula}	(pendiente) Actualizar cliente
DELETE	/clientes/{cedula}	(pendiente) Eliminar cliente
GET	/habitaciones	Listar todas las habitaciones
POST	/habitaciones	Agregar nueva habitación
GET	/habitaciones/{id}	Consultar habitación por número
PUT	/habitaciones/{id}	Actualizar tipo, precio o disponibilidad

Método	URL	Descripción
DELETE	/habitaciones/{id}	<i>(pendiente)</i> Eliminar habitación
GET	/habitaciones/disponibles-premium	<i>(pendiente)</i> Habitaciones sobre el precio promedio (subconsulta)
GET	/reservas	Listar reservas (SELECT * FROM Reservar)
POST	/reservas	Crear reserva (dispara trg_no_solapamiento)
PUT	/reservas/{id}	Actualizar reserva (dispara trg_no_solapamiento)
POST	/reservas/solicitar	Registrar solicitud de servicio
DELETE	/reservas/{id}	<i>(pendiente)</i> Cancelar reserva
GET	/empleados	Listar todos los empleados
POST	/empleados	Registrar empleado (dispara trg_audit_empleado)
GET	/empleados/telefonos	Listar teléfonos de empleados
POST	/empleados/telefonos	Registrar teléfono de empleado
GET	/empleados/{cedula}/telefonos	Teléfonos de un empleado por cédula
POST	/empleados/areas	Crear nueva área de trabajo
POST	/empleados/servicios	Crear nuevo servicio en catálogo
PUT	/empleados/{cedula}	<i>(pendiente)</i> Actualizar empleado
DELETE	/empleados/{cedula}	<i>(pendiente)</i> Eliminar empleado

Cuadro 12: Endpoints de la API REST (implementados y pendientes)